

Predictive Analytics for Health Risk Stratification: Machine Learning Applications in Non-Hospital COVID-19 Treatment Eligibility



Sriharsha Surannagari (W9607526)

Supervisor: Dr Mansha Nawaz

School of Computing, Engineering and Digital Technologies
Teesside University, UK

The dissertation is submitted for the degree of
MSc in Data Science

January 7, 2025

Declaration

I hereby confirm that, except where specific references to others' work have been made, all content of my dissertation is original and has not been submitted in whole or part for consideration for any degree or qualification at this university or any other. Furthermore, this dissertation represents my efforts alone and contains nothing which was done collaboratively unless specified within its text and acknowledgements.

Sriharsha Surannagari (W9607526)

January 7, 2025

Acknowledgements

I express my deepest gratitude to my supervisor, Dr. Mansha Nawaz, for his invaluable support, guidance, and encouragement throughout this project. His guidance, expertise, and mentorship provided me with a valuable direction and were instrumental in achieving the desired outcomes. I would also like to extend my sincere thanks to my family for their support, encouragement throughout this journey. Finally, I am grateful to Teesside University for providing me with the opportunity, resources to successfully complete this research.

Abstract

This dissertation investigates the use of machine learning techniques in healthcare, specifically to identify high-risk individuals eligible for non-hospital COVID-19 treatment. Using publicly available datasets from NHS, which contain demographic and clinical information on 1.49 million digitally identified high-risk individuals, the study uses machine learning algorithms to analyze and predict treatment eligibility. Preprocessing steps such as data cleaning, imputation, feature engineering and class balancing used to enhance the quality and reliability of this analysis. Key findings demonstrate that machine learning models such as gradient boosting regressor and AdaBoost classifier successfully identify high-risk patients with robust performance. The gradient boosting regressor achieved a R^2 of 0.2454, MSE of 0.1657, and RMSE of 0.4071, while AdaBoost classifier recorded perfect accuracy scores for accuracy (1.0), precision (1.0), recall (1.0), and F1-score (1.0). A generalized model devised to avoid overfitting or underfitting, this allowed consistent performance across training and test datasets. These results demonstrate how machine learning can assist in providing more accurate risk stratification for non-hospital COVID-19 treatment, facilitating timely interventions and more efficient resource allocation.

Table of contents

List of figures	vi
List of tables	ix
0.1 Introduction	1
0.1.1 Problem Statement	1
0.1.2 Research Question and Objectives	1
0.1.3 Objectives	1
0.1.4 Structure of the Report	2
0.2 Literature Review	2
0.3 Methodology	5
0.4 Dataset Summary and Workflow	5
0.4.1 Dataset Summary	5
0.4.2 Workflow	6
0.4.3 Power BI Dashboards	26
0.5 Ethical Consideration	29
0.6 Results	29
0.6.1 Classification	29
0.6.2 Regression	31
0.6.3 Performance Analysis	33
0.7 Conclusion	39
References	41

List of figures

1	Flowchart of the methodology for evaluating machine learning models in health risk stratification.	7
2	Data loading and preprocessing workflow in Google Colab for classification.	8
3	Data loading and preprocessing workflow in Google Colab for regression. .	8
4	Code snippet displaying the computation of summary statistics and checking for missing values.	9
5	Summary statistics and data quality analysis showing complete data across demographic variables including age groups, gender, and ethnic categories.	10
6	Code snippet illustrating numerical feature selection and Exploratory Data Analysis (EDA) steps such as plotting distributions and generating correlation heatmaps.	11
7	Distribution plots with kernel density estimation curves revealing population patterns and potential outliers across demographic variables.	11
8	Distribution of the <i>Total</i> variable, highlighting its right-skewed nature. . . .	12
9	Correlation heatmap displaying strong positive relationships between demographic variables, with darker colors indicating stronger correlations.	12
10	Summary statistics of numerical features, including count, mean, standard deviation, and percentiles.	13
11	Feature engineering: renaming columns, creating a binary target variable, and selecting features by dropping demographics.	14
12	Implementation of SMOTE algorithm showing the balancing of the feature set and target variable using a synthetic sample generation approach with k-nearest neighbors.	14
13	Implementation of train-test split showing 80:20 data division using scikit-learn's <code>train_test_split</code> function with balanced feature set and target variable.	15
14	Implementation of multiple classification models including Logistic Regression, Random Forest (RF), Gradient Boosting (GB), Decision Tree, AdaBoost (AB), and SVM with specified parameters.	15

15	Implementation of multiple regression models including Linear Regression (LR), RF Regressor, GB Regressor, and Decision Tree Regressor with specified parameters.	15
16	Grid search cross-validation for classification models, optimizing parameters for RF and GB classifiers.	16
17	Grid search cross-validation for regression models, optimizing parameters for RF, GB, and Decision Tree (DT) regressors.	17
18	Implementation of model evaluation for classification models, including metric calculation (accuracy, precision, recall, F1-score, Area Under the Curve - Receiver Operating Characteristic (AUC-ROC)) and confusion matrix generation.	19
19	Implementation of model evaluation for regression models, showing metric calculation (Mean Square Error (MSE), Root Mean Square Error (RMSE), R-squared) for assessing performance.	20
20	Implementation of model evaluation for regression models, showing metric calculation (MSE, RMSE, R-squared) for assessing performance.	20
21	calculating classification metrics (Accuracy, Precision, Recall, F1-Score) for both training and testing datasets	21
22	Code snippet for training and evaluating regression models, including the calculation of train and test metrics (MSE, RMSE, and R^2) for each model.	22
23	Visualizing classification metrics (Accuracy, Precision, Recall, F1-Score, and AUC-ROC) and confusion matrices.	23
24	visualizing classification metrics (Accuracy, Precision, Recall, F1-Score) using bar plots, comparing training and testing performance across models, with annotations for metric values and legends for clarity.	24
25	Visualizing Regression metrics (MSE, RMSE, R^2)	25
26	Bar plots comparing training and testing regression metrics across models	26
27	Power BI visualization summarizing gender, age group, and clinical condition distributions.	28
28	Power BI visualization illustrating trends of clinical conditions across age groups.	28
29	Comparison of classification metrics (accuracy, precision, recall, and F1-score) across models. AB outperforms all other models.	30
30	Confusion matrices for classification models, showing true positives, false positives, true negatives, and false negatives. AB achieves perfect classification, whereas Logistic Regression performs poorly.	31

31	Comparison of MSE across regression models.	32
32	Comparison of RMSE across regression models.	32
33	Comparison of R^2 values across regression models.	33
34	Train vs Test Accuracy Comparison Across Classification Models	34
35	Train vs Test Precision Comparison Across Classification Models	35
36	Train vs Test Recall Comparison Across Classification Models	35
37	Train vs Test F1-Score Comparison Across Classification Models	36
38	Train vs Test MSE Comparison Across Regression Models.	37
39	Train vs Test RMSE Comparison Across Regression Models.	38
40	Train vs Test R^2 Comparison Across Regression Models.	38

List of tables

1	Machine Learning Models and Their Applications in Healthcare	3
2	Dataset Summary	6
3	Classification Results for Health Risk Stratification Models	30
4	Regression Results for Health Risk Stratification Models	31
5	Performance Metrics for Classification Models	34
6	Regression Model Performance Metrics	37

0.1 Introduction

The COVID-19 pandemic has highlighted the urgent need for advanced methodologies that identify high-risk individuals quickly and offer timely, targeted interventions [1, 2]. As a response, the UK Government has used data-driven approaches to digitally identify individuals at highest risk from COVID-19 complications with non-hospital-based treatment options [2]. These efforts illustrate the essential role predictive analytics plays in optimizing healthcare resource allocation and guaranteeing equitable access for vulnerable populations [3]. Machine Learning (ML) one of the core areas of artificial intelligence, provides an efficient framework for the analysis of large, high-dimensional datasets. Healthcare applications of this technique include predictive modeling, risk stratification and resource optimization which facilitate timely interventions and ultimately enhance both patient care and system efficiency [4, 5].

0.1.1 Problem Statement

Though ML has advanced quickly in recent years, significant challenges still exist when applied to healthcare. Identification of high-risk individuals within healthcare systems like National Health Service (NHS) is still limited by issues like poor data quality and class imbalance as well as interpretability of predictive models. Chronic conditions like Chronic Obstructive Pulmonary Disease (COPD), diabetes and heart disease require robust models with adaptable predictive capabilities to facilitate timely interventions and limit hospital admissions, but many current approaches fail to produce actionable insights due to difficulty aligning with real-life healthcare scenarios.

0.1.2 Research Question and Objectives

Research Question

How can machine learning models be applied to NHS healthcare data to accurately identify high-risk patients, thus facilitating timely interventions and minimizing hospital admissions?

0.1.3 Objectives

- **Investigate Current Methods:** Investigate existing healthcare strategies and techniques used for identifying high-risk patients, with particular attention paid to machine learning applications.

- **Data Analysis and Preprocessing:** Extract, clean and preprocess relevant features from NHS Digital datasets to produce high-quality data for model training purposes.
- **Develop Predictive Models:** Implement machine learning algorithms like logistic regression, RFs and neural networks to build models capable of accurately predicting high-risk patients.
- **Evaluate Model Performance:** Assess models using performance metrics such as accuracy, sensitivity, specificity, precision, recall and RMSE, MSE and R^2 to comprehensively assess their reliability and robustness.
- **Validate Models with Real-World Data:** Validate machine learning models on NHS datasets in real-time to make sure that they can adapt effectively to previously unknown information.
- **Recommend Integration Strategies:** Provide actionable recommendations for incorporating developed models into NHS workflows to optimize decision-making and patient care.
- **Address Ethical Considerations:** Make sure that all data handled according to NHS and GDPR guidelines, with an emphasis on fairness, transparency, and accurate predictions.

0.1.4 Structure of the Report

This report is organized into five main sections. The Introduction presents background, problem statement, research aim and objectives as well as an outline for this report. The Literature Review reviews current research into machine learning techniques used in healthcare settings, while Methodology details the processes involved with data preprocessing, model development, evaluation metrics and ethical considerations, Results present findings from model implementation and evaluation, while finally the Conclusion summarizes key findings, contributions made and future directions suggested for further study.

0.2 Literature Review

A growing interest in the integration of ML, into health-care is due to its potential for transforming patient care and resource allocation. This section gives an overview of the existing literature regarding ML, their applications in health care, and challenges associated

with implementation. The study also examines the key metrics and methods used to evaluate and assess the performance of these models.

ML can generally be divided into three major approaches: supervised learning, unsupervised learning and reinforcement learning as shown in Table 1.

Table 1 Machine Learning Models and Their Applications in Healthcare

Type of ML Model	Model	Applications in Healthcare
Supervised Learning [6]	LR	Predicting health outcomes such as survival rates or disease progression.
	RF	Identification of risk factors, diagnosis of conditions and prediction of readmission probabilities.
	GB	Classifying high-risk patient groups for diseases like diabetes.
	SVM	Disease classification (such as cancer subtypes) and patient outcome prediction.
Unsupervised Learning [6, 7]	K-Means	Patient segmentation and cluster identification for diagnosis.
	PCA	Dimensionality reduction for medical imaging or health records.
	Autoencoders	Anomalous detection in medical images and health data.
Ensemble Learning [7]	Stacking	Combining model predictions for hospital stay forecasting.
	Bagging	Robust chronic disease risk and adherence prediction models.
	AB	Enhancing prediction accuracy for disease progression.
Deep Learning [8]	CNN	Medical image analysis (e.g., tumor detection).
	RNN	Enhancing prediction accuracy of disease progression.
Reinforcement Learning [6]	Q-Learning	Optimizing treatment strategies for chronic diseases.
	DQN	Customizing treatment plans dynamically.

Supervised learning entails training models on labeled datasets to predict outcomes such as disease progression or patient survival rates, making it the foundation of predictive

analytics in healthcare settings [6]. Unsupervised learning identifies hidden patterns within unlabeled data to reveal hidden trends, which can then be applied towards clustering patient subgroups or detecting anomalies within health records [6]. Reinforcement learning aims to maximize sequential decision-making through trial-and-error and has been used successfully in applications like resource management and personalized treatment planning. Advanced machine learning techniques such as ensemble learning and deep learning extend its capabilities even further, enabling the creation of model predictions as well as analysis of complex datasets such as medical imaging images [7, 8]. These methodologies highlight the ability of ML to meet diverse challenges within healthcare. Addressing challenges such as missing data, class imbalance, and variable feature scales is important to ensuring accurate machine learning models in healthcare applications [9, 10]. Missing data can create bias or inconsistency in predictions, imputation techniques attempt to estimate and fill these gaps for full dataset completion. Class imbalance, where one class dominates over others in a dataset, may result in biased predictions favoring that majority class [9, 10]. Synthetic Minority Over-sampling Technique (SMOTE) is widely employed to balance datasets by creating synthetic samples of minorities to ensure fair and accurate predictions for data sets with imbalanced sampling [11–13]. Normalization, the practice of standardizing feature values, is essential for aligning datasets with various attributes and providing consistent model training to enhance interpretability [14–17]. Evaluating ML models requires understanding key concepts like underfitting, overfitting and generalization. Underfitting occurs when models fail to recognize underlying patterns in data which leads to poor performance on both training and test datasets, overfitting occurs when models learn noise or specific patterns within training data which results in high accuracy but poor generalization to unseen data sets. Generalization measures the ability of models to adapt well to new, unseen input, it's an indicator of robustness [18–20]. Metrics such as Accuracy, Precision, Recall and F1-Score generated from confusion matrix components like True Positives - Correctly predicted positive instances (TP), False Positives - Incorrectly predicted positive instances (Type I error) (FP), True Negatives - Correctly predicted negative instances (TN) and False Negatives - Incorrectly predicted negative instances (Type II error) (FN) provide useful metrics for evaluating sensitivity and specificity in imbalanced datasets also RMSE, MSE and acR^2 integral for evaluating the reliability and performance of regression models [21–24].

Studies reviewed highlight the significant role machine learning ML can play in solving key healthcare challenges, from predictive modelling to resource optimization. Existing literature emphasizes the significance of dealing with data challenges such as missing values, class imbalance, and variable feature scales to ensure robust interpretable models are developed. Furthermore, advanced ML techniques such as ensemble learning, deep learning

and reinforcement learning have proved greatly effective at improving patient care as well as system efficiency.

However, despite these advancements in machine learning for health risk stratification applications such as using publicly available datasets to address chronic disease management challenges within systems like NHS. This research seeks to address this gap by creating predictive models which identify high-risk individuals and provide actionable insights for proactive patient care. The next section presents methodology used.

0.3 Methodology

This study uses Python to systematically evaluate machine learning models for health risk stratification, while Power Business Intelligence (PowerBI) dashboards were then utilized to visualize key data trends, including demographic information, clinical condition prevalence rates and age-specific patterns. Python is used for model development as shown in Figure 1, followed by visualizing and summarizing them via PowerBI dashboards as shown in Figures 27 and 28.

0.4 Dataset Summary and Workflow

0.4.1 Dataset Summary

Table 2 provides an overview of the dataset, with column names and data types and descriptions [2].

Table 2 Dataset Summary

Column Name	Data Type	Description
Total	int64	Total count of individuals
Female	int64	Count of females in the dataset
Male	int64	Count of males in the dataset
12 to 16	int64	Age group: 12 to 16
17 to 19	int64	Age group: 17 to 19
20 to 29	int64	Age group: 20 to 29
30 to 39	int64	Age group: 30 to 39
40 to 49	int64	Age group: 40 to 49
50 to 59	int64	Age group: 50 to 59
60 to 69	int64	Age group: 60 to 69
70 to 79	int64	Age group: 70 to 79
80 to 89	int64	Age group: 80 to 89
90+	int64	Age group: 90 and above
Asian	int64	Ethnicity: Asian/Asian British
Black	int64	Ethnicity: Black/African/Caribbean/Black British
Mixed	int64	Ethnicity: Mixed/Multiple ethnic groups
Not Stated	int64	Ethnicity: Not Stated
Other	int64	Ethnicity: Other ethnic groups
White	int64	Ethnicity: White
Condition Group	object	Clinical condition group (categorical)

0.4.2 Workflow

Figure 1 illustrates the methodological workflow utilized for health risk stratification in this research study.

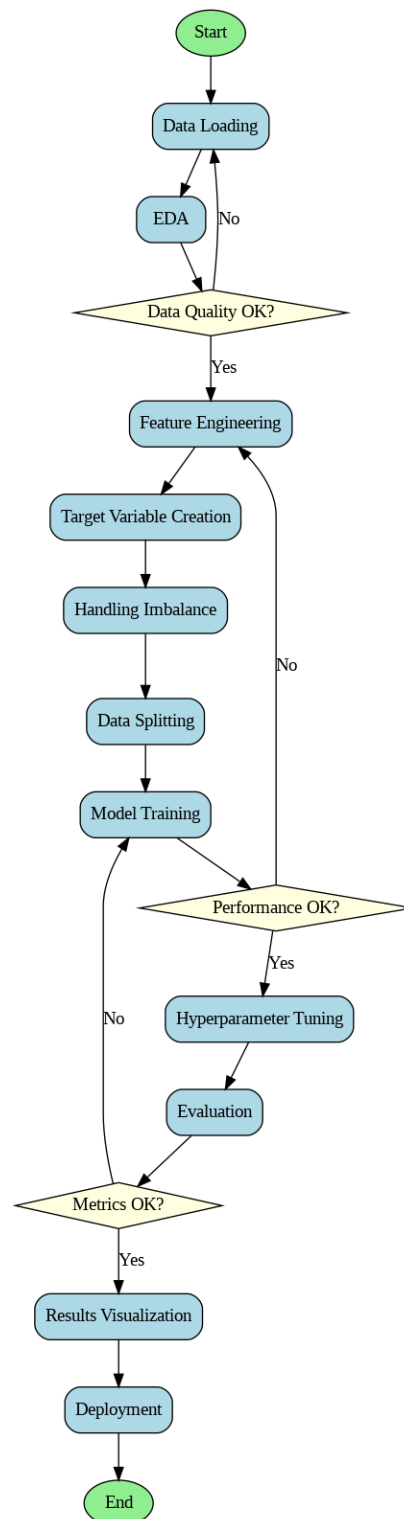


Fig. 1 Flowchart of the methodology for evaluating machine learning models in health risk stratification.

1. **Data Loading:** The proposed model's workflow starts with data loading and initial setup in Google Colab and jupyter notebook. Essential Python libraries such as pandas, numpy, matplotlib and seaborn were utilized for efficient data manipulation, numerical computations and visualization tasks, scikit-learn used for various machine learning tasks including training evaluation preprocessing, the dataset loaded using `pandas.read_csv` and column names standardized before processing steps being undertaken. Figures 2 and 3 show workflows for loading and preprocessing data used for classification and regression tasks.

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV

import seaborn as sns

# Load dataset
data = pd.read_csv("digi-iden-high-risk-covi-non-hosp-oct22.csv")
```

Fig. 2 Data loading and preprocessing workflow in Google Colab for classification.

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV

# Step 1: Load and preprocess the data
data = pd.read_csv("digi-iden-high-risk-covi-non-hosp-oct22.csv")
```

Fig. 3 Data loading and preprocessing workflow in Google Colab for regression.

2. EDA and Data Quality

EDA provide a thorough examination of the characteristics and quality of the dataset. Initial checks confirmed there were no missing values across any 20 variables verified using `isnull().sum()` function. The dataset included a population range from 30,314 to 1,488,765, with females constituting more than 60% (mean: 181,601) than males (164,432). Age demographics were predominantly concentrated among the 70-79 age demographic (mean 85 182) while the White population (mean 294,524) was the predominant ethnic group followed by Asian (19 4477) and Black (17169) populations. Figure 4 shows the computation and testing for missing values using summary statistics.

Figure 5 shows data quality assessments through summary statistics and quality checks. Figure 6 provides details of numerical feature selection, plotting distributions and creating a correlation heatmap. Population patterns, potential outliers and distribution variables can be visualized using kernel density estimation as shown in Figure 7. Figure 8 shows a highly right-skewed distribution for the Total, emphasizing the necessity of normalization [17], log transformation before model training to ensure equal results.

```
# Display summary statistics
print("Summary Statistics:")
print(data.describe())

# Check for missing values
print("\nMissing Values:")
print(data.isnull().sum())
```

Fig. 4 Code snippet displaying the computation of summary statistics and checking for missing values.

```

Summary Statistics:
      Total      Female      Male      12 to 16      17 to 19 \
count  1.100000e+01      11.000000      11.000000      11.000000      11.000000
mean   3.540621e+05  189601.818182  164432.181818  3796.181818  2625.000000
std    4.507134e+05  242088.497897  209385.217254  4959.784730  3445.109606
min    3.031400e+04  16371.000000  11944.000000  139.000000  116.000000
25%    1.081325e+05  47079.000000  48375.500000  1125.500000  692.000000
50%    1.684130e+05  97042.000000  79927.000000  2497.000000  1626.000000
75%    3.697205e+05  209649.500000  161763.500000  3866.500000  2674.000000
max    1.488765e+06  792553.000000  696074.000000  17546.000000  11837.000000

      20 to 29      30 to 39      40 to 49      50 to 59 \
count  11.000000  11.000000  11.000000  11.000000
mean   12264.272727  23377.000000  34407.363636  58091.181818
std    16411.590502  31427.815075  45571.378518  73819.003548
min    674.000000  2691.000000  5413.000000  4853.000000
25%    3493.500000  6308.500000  10057.500000  21350.000000
50%    5375.000000  8278.000000  13629.000000  28527.000000
75%    12089.000000  22094.500000  32167.500000  55286.500000
max    54680.000000  104669.000000  152548.000000  248465.000000

      60 to 69      70 to 79      80 to 89      90+ \
count  11.000000  11.000000  11.000000  11.000000
mean   71789.272727  85182.454545  51583.000000  10946.363636
std    88542.314442  107507.496856  68000.763477  15399.099300
min    2283.000000  433.000000  82.000000  10.000000
25%    27014.500000  21066.500000  7824.500000  1171.500000
50%    32067.000000  49062.000000  20307.000000  1959.000000
75%    79438.500000  103266.500000  71680.500000  18539.000000
max    293091.000000  342752.000000  214291.000000  48886.000000

      Asian      Black      Mixed      Not_Stamped      Other \
count  11.000000  11.000000  11.000000  11.000000  11.000000
mean   19447.909091  17169.000000  4599.545455  13496.818182  4824.363636
std    24103.813169  24643.057201  6158.503574  17402.163146  6235.943125
min    1937.000000  1231.000000  658.000000  1115.000000  585.000000
25%    5462.000000  4161.500000  1673.000000  3712.500000  1599.000000
50%    8811.000000  6655.000000  2059.000000  6382.000000  2676.000000
75%    22676.500000  15077.000000  3659.500000  14235.500000  3901.000000
max    82246.000000  81961.000000  20808.000000  56879.000000  21311.000000

      White
count  1.100000e+01
mean   2.945245e+05
std    3.742492e+05
min    4.182000e+03
25%    8.804550e+04
50%    1.465460e+05
75%    3.201000e+05
max    1.225560e+06

Missing Values:
Condition_Group  0
Total           0
Female          0
Male            0
12 to 16        0
17 to 19        0
20 to 29        0
30 to 39        0
40 to 49        0
50 to 59        0
60 to 69        0
70 to 79        0
80 to 89        0
90+             0
Asian           0
Black           0
Mixed           0
Not_Stamped     0
Other           0
White           0
dtype: int64

```

Fig. 5 Summary statistics and data quality analysis showing complete data across demographic variables including age groups, gender, and ethnic categories.

```

# Select numerical features
numerical_features = data.select_dtypes(include=['int64', 'float64']).columns

# Compute summary statistics for numerical features
summary_stats = data[numerical_features].describe()

# Plot distributions of numerical features
plt.figure(figsize=(12, 8))
for i, col in enumerate(numerical_features, 1):
    plt.subplot(int(np.ceil(len(numerical_features)/4)), 4, i)
    sns.histplot(data[col], kde=True, bins=20, color="skyblue")
    plt.title(f"{col}")
plt.tight_layout()
plt.savefig("numerical_distributions.png")
plt.show()

# Plot correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(data[numerical_features].corr(), annot=True, fmt=".2f")
plt.title("Correlation Heatmap")
plt.savefig("correlation_heatmap.png")
plt.show()

# Prepare summary statistics for plotting
stats_to_plot = ['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max']
summary_stats_to_plot = summary_stats.loc[stats_to_plot].transpose()

# Create a bar plot for summary statistics
fig, ax = plt.subplots(figsize=(24, 12))
summary_stats_to_plot.plot(kind='bar', width=0.8, colormap='tab10', ax=ax)

# Customize plot aesthetics
ax.set_title("Summary Statistics of the Numerical Features", fontsize=24, weight='bold')
ax.set_xlabel("Numerical Features", fontsize=20, weight='bold')
ax.set_ylabel("Value", fontsize=20, weight='bold')
ax.tick_params(axis='x', labelrotation=60, labelsz=14)
ax.tick_params(axis='y', labelsz=16)
ax.legend(title="Statistics", bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=14, title_fontsize=16)

# Adjust layout and save the plot
plt.tight_layout()
plt.savefig("summary_statistics_plot_fixed.png", dpi=300, bbox_inches='tight')
plt.show()

```

Fig. 6 Code snippet illustrating numerical feature selection and EDA steps such as plotting distributions and generating correlation heatmaps.

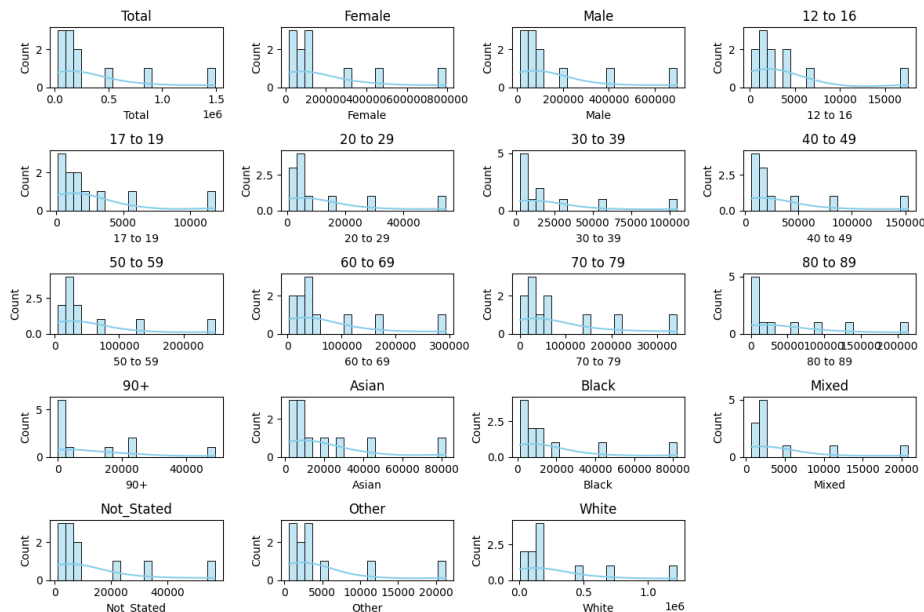


Fig. 7 Distribution plots with kernel density estimation curves revealing population patterns and potential outliers across demographic variables.

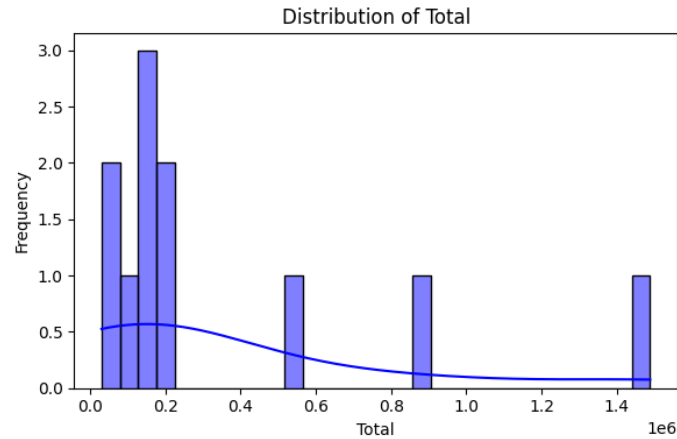


Fig. 8 Distribution of the *Total* variable, highlighting its right-skewed nature.

Correlation analysis demonstrated in Figure 9 shows significant positive correlations (greater than 0.90) among several demographic variables, specifically age groups and ethnic categories. This highlights feature relationships, which are essential in helping guide feature selection decisions as well as reduce multicollinearity in machine learning models.

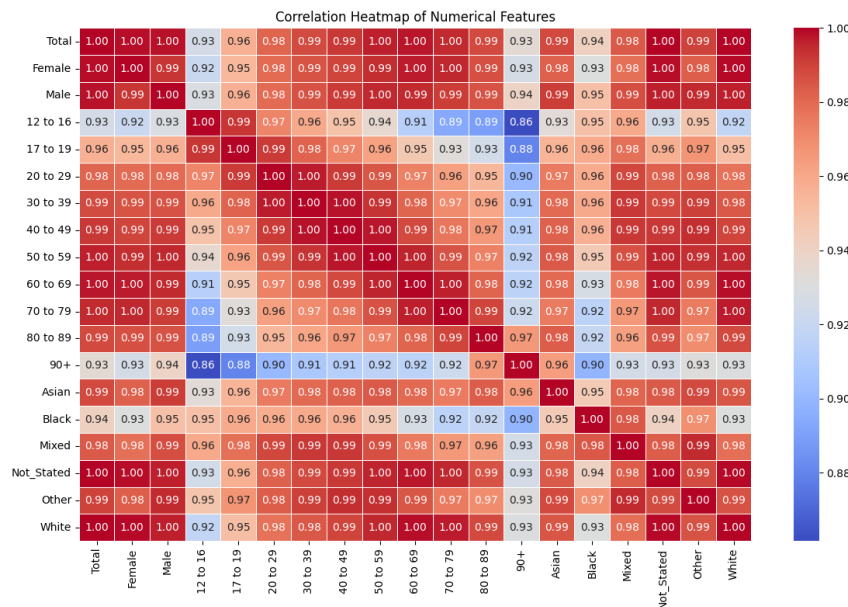


Fig. 9 Correlation heatmap displaying strong positive relationships between demographic variables, with darker colors indicating stronger correlations.

Numerical feature analysis provided insights into central tendencies and variability, as shown in Figure 10.

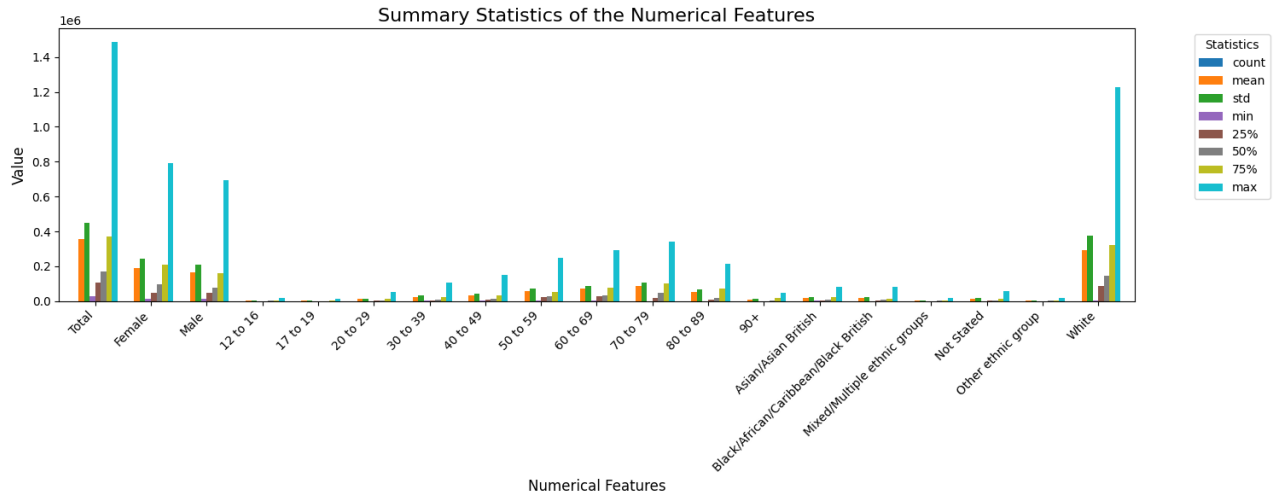


Fig. 10 Summary statistics of numerical features, including count, mean, standard deviation, and percentiles.

3. Feature Engineering and Target Variable Creation:

Figure 11 shows three steps involved in feature engineering. First, column names were simplified by streamlining ethnic group categories into simpler ones, such as changing 'Asian/Asian British' to simply 'Asian' for easier data handling purposes and "Black/African/Caribbean/Black British" into simply "Black". Created a binary target variable HighRisk using the median value of the Total column, values above this median were classified as high risk while those below qualified as low risk (0). This binary target variable was applied for classification and regression tasks to ensure consistency in modeling. Finally, feature engineering performed to form a focused feature set for model training.

```

# Rename columns for clarity and consistency
data.rename(columns={
    'Asian/Asian British': 'Asian',
    'Black/African/Caribbean/Black British': 'Black',
    'Mixed/Multiple ethnic groups': 'Mixed',
    'Not Stated': 'Not_Stated',
    'Other ethnic group': 'Other',
    'Clinical condition group': 'Condition_Group'
}, inplace=True)

# Create binary target variable for classification
data['High_Risk'] = (data['Total'] > data['Total'].median()).astype(int)

# Feature Engineering
X_features = data.drop(columns=['Condition_Group', 'Total', 'High_Risk', 'Female', 'Male',
                                '12 to 16', '17 to 19', '20 to 29', '30 to 39', '40 to 49',
                                '50 to 59', '60 to 69', '70 to 79', '80 to 89', '90+'])
X = X_features
y = data['High_Risk']

```

Fig. 11 Feature engineering: renaming columns, creating a binary target variable, and selecting features by dropping demographics.

4. Handling Class Imbalance:

SMOTE algorithm used to address class imbalance, applicable both classification and regression tasks. SMOTE is initialized with `random_state=42` for reproducibility and `k_neighbors=1` for synthetic sample generation, after which, its application balanced both feature set (X) and target variable (y), creating versions `X_balanced` and `y_balanced` respectively ensuring equal representation of classes in the dataset and improving performance for minorities for both tasks as shown in Figure 12 .

```

# Handle class imbalance using SMOTE
smote = SMOTE(random_state=42, k_neighbors=1)
X_balanced, y_balanced = smote.fit_resample(X, y)

```

Fig. 12 Implementation of SMOTE algorithm showing the balancing of the feature set and target variable using a synthetic sample generation approach with k-nearest neighbors.

5. **Data Splitting:** Scikit-learn's `train_test_split` function is used to split the data set into training and testing sets using an 80:20 split ratio as shown in Figure 13. The split was performed on a balanced dataset (`X_balanced`, `y_balanced`) obtained after the SMOTE application with `random_state=42` for reproducibility to ensure consistent class distributions between training and testing sets and maintain their balance for model training and evaluation purposes as shown in Figure 13.

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_balanced, y_balanced, test_size=0.2, random_state=42)
```

Fig. 13 Implementation of train-test split showing 80:20 data division using scikit-learn's `train_test_split` function with balanced feature set and target variable.

6. Model Training:

Multiple models for classification and regression tasks were implemented and trained using scikit-learn library. Models implemented for classification tasks included Logistic Regression with maximum iterations set at 1000, RF classifier, GB classifier, Decision Tree classifier, AB classifier and Support Vector Machines (SVM) with probability estimation enabled, while for regression tasks LR, RF regressor, GB regressor and Decision Tree regressor included. Figures 14 and 15 illustrate their implementation respectively.

```
# Define models for classification
classification_models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "SVM": SVC(probability=True, random_state=42)
}
```

Fig. 14 Implementation of multiple classification models including Logistic Regression, RF, GB, Decision Tree, AB, and SVM with specified parameters.

```
# Define models for regression
regression_models = {
    "Linear Regression": LinearRegression(),
    "Random Forest Regressor": RandomForestRegressor(random_state=42),
    "Gradient Boosting Regressor": GradientBoostingRegressor(random_state=42),
    "Decision Tree Regressor": DecisionTreeRegressor(random_state=42)
}
```

Fig. 15 Implementation of multiple regression models including LR, RF Regressor, GB Regressor, and Decision Tree Regressor with specified parameters.

7. Hyperparameter Tuning:

Grid search was used to optimize both classification and regression models by testing various hyperparameter combinations systematically. RF classifier parameters such as

a number of estimators (n_estimators: 100, 200) and maximum depth (max_depth: 10, 20) optimized. GB classifier also utilized grid search in this manner with its number of estimators (n_estimators: 100, 200) and learning rate (learning_rate: 0.01, 0.1) being optimized. The implementation for classification is shown in Figure 16.

```
# Define parameter grids for hyperparameter tuning
param_grids = {
    "Random Forest": { # Parameter grid for Random Forest
        'n_estimators': [100, 200], # Number of trees in the forest
        'max_depth': [10, 20] # Maximum depth of each tree
    },
    "Gradient Boosting": { # Parameter grid for Gradient Boosting
        'n_estimators': [100, 200], # Number of boosting stages
        'learning_rate': [0.01, 0.1] # Step size for updating weights
    }
}

# Iterate through the models and their parameter grids
for name, model in classification_models.items():
    if name in param_grids: # Check if the model has a defined parameter grid
        # Initialize GridSearchCV with model, parameter grid, and 5-fold cross-validation
        grid_search = GridSearchCV(model, param_grids[name], cv=5)

        # Perform hyperparameter tuning by fitting the model to the training data
        grid_search.fit(X_train, y_train)

        # Retrieve and print the best hyperparameters for the model
        print(f"Best parameters for {name}: {grid_search.best_params_}")
        print(f"Best cross-validated score for {name}: {grid_search.best_score_}")
```

Fig. 16 Grid search cross-validation for classification models, optimizing parameters for RF and GB classifiers.

Regression models optimized by tuning the RF regressor's parameters including several estimators (n_estimators: 100, 200) and maximum depth (max_depth: 10, 20). GB regressor parameters such as several estimators (n_estimators: 100, 200) and learning rate (learning_rate: 0.01, 0.1) tuned. The DT regressor further optimized by its maximum depth (max_depth: 10, 20) and the minimum number of samples required to split a node min_samples_split: 2, 5) with grid search process as shown in Figure 17.

```
# Define hyperparameter grids for regression models
param_grids = {
    "Random Forest Regressor": {
        'n_estimators': [100, 200],
        'max_depth': [10, 20]
    },
    "Gradient Boosting Regressor": {
        'n_estimators': [100, 200],
        'learning_rate': [0.01, 0.1]
    },
    "Decision Tree Regressor": {
        'max_depth': [10, 20],
        'min_samples_split': [2, 5]
    }
}
```

Fig. 17 Grid search cross-validation for regression models, optimizing parameters for RF, GB, and DT regressors.

8. Model Evaluation:

The evaluation process included comprehensive performance assessments for classification and regression models, as detailed in Figures 18 and 19.

Classification models were evaluated according to multiple metrics, such as accuracy, precision, recall, F1-score and AUC-ROC [25].

- **TP:** Correctly predicted positive instances
- **TN:** Correctly predicted negative instances
- **FP:** Incorrectly predicted positive instances (Type I error)
- **FN:** Incorrectly predicted negative instances (Type II error)

The confusion matrix for a binary classification problem [25] is represented as:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Accuracy: Accuracy measures the overall correctness of the model's predictions.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Accuracy measures the percentage of correct predictions (both true positives and false negatives) made among all cases examined.

Precision: Precision measures the accuracy of positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision measures the ratio of correctly predicted positive observations relative to total predicted positive observations.

Recall: Recall measures the model's ability to find all positive instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall, also referred to as sensitivity or true positive rate, measures the ratio between accurately predicted positive observations and actual positive observations.

F1-Score: The F1-score is the harmonic mean of precision and recall, providing a single score that balances both metrics.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-score provides single score that balances precision and recall.

AUC-ROC: The AUC-ROC curve is a performance measurement for classification problems at various threshold settings.

A ROC curve plots True Positive Rate (TPR) against F (FP) rate, at various threshold settings:

$$TP_rate = \frac{TP}{TP + FN}$$

$$FP_rate = \frac{FP}{FP + TN}$$

AUC-ROC measures the degree or measure of separability between classes. This number ranges from 0-1 with 1 representing perfect separation.

- AUC = 1: Denotes perfect classification.
- AUC = 0.5: Denotes performance equivalent to random guessing.
- AUC < 0.5: Denotes performance suboptimal to random guessing.

Results were stored in an organized format and confusion matrices were produced to gain more in-depth insights into classification performance classes as shown in Figure 18.

```
# Model evaluation implementation

classification_results = [] # Initialize a list to store evaluation results for each model
confusion_matrices = {}    # Initialize a dictionary to store confusion matrices for each model

for name, model in classification_models.items(): # Iterate through the classification models
    # Train the model using the training data
    model.fit(X_train, y_train)

    # Generate predictions on the test data
    y_pred = model.predict(X_test) # Predicted class labels
    y_prob = model.predict_proba(X_test)[:, 1] # Predicted probabilities for the positive class (1)

    # Generate a classification report (includes metrics like precision, recall, F1-score, accuracy)
    report = classification_report(y_test, y_pred, output_dict=True)

    # Calculate the Area Under the Receiver Operating Characteristic Curve (AUC-ROC)
    auc_roc = roc_auc_score(y_test, y_prob)

    # Append the evaluation metrics for the current model to the results list
    classification_results.append({
        "Model": name, # Model name
        "Accuracy": report["accuracy"], # Overall accuracy
        "Precision": report["1"]["precision"], # Precision for the positive class
        "Recall": report["1"]["recall"], # Recall (sensitivity) for the positive class
        "F1-Score": report["1"]["f1-score"], # F1-score for the positive class
        "AUC-ROC": auc_roc # AUC-ROC score
    })

    # Compute and store the confusion matrix for the current model
    confusion_matrices[name] = confusion_matrix(y_test, y_pred)
```

Fig. 18 Implementation of model evaluation for classification models, including metric calculation (accuracy, precision, recall, F1-score, AUC-ROC) and confusion matrix generation.

Regression models were evaluated using metrics like MSE, RMSE, and R^2 .

These metrics measure predictive accuracy and can be described as follows:

- **MSE:**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

where y_i represents the observed values, $\hat{y}_i$ represents the predicted values, and n is the total number of data points.

- **RMSE:**

$$\text{RMSE} = \sqrt{\text{MSE}},$$

which provides the square root of the average squared prediction error, making it interpretable in the same units as the target variable.

The results were stored systematically, and an evaluation process ensured the identification of the most efficient regression models by analyzing error metrics as shown in Figures 19, 20.

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Initialize a list to store evaluation results for each regression model
regression_results = []

# Initialize a dictionary to store predictions for each model for further analysis
regression_predictions = {}

# Iterate through the regression models
for name, model in regression_models.items():
    print(f"Evaluating {name}...") # Log the model being evaluated

    # Train the model using the training data
    model.fit(X_train, y_train)

    # Generate predictions on the test data
    y_pred = model.predict(X_test)

    # Calculate regression evaluation metrics
    mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
    rmse = np.sqrt(mse) # Root Mean Squared Error
    r2 = r2_score(y_test, y_pred) # R-squared score

    # Append the evaluation metrics for the current model to the results list
    regression_results.append({
        "Model": name, # Model name
        "MSE": mse, # Mean Squared Error
        "RMSE": rmse, # Root Mean Squared Error
        "R2": r2 # R-squared score
    })

    # Store predictions for analysis
    regression_predictions[name] = y_pred

# Output the final regression evaluation results
print("Regression evaluation results:")
for result in regression_results:
    print(result)
```

Fig. 19 Implementation of model evaluation for regression models, showing metric calculation (MSE, RMSE, R-squared) for assessing performance.

```
Evaluating Linear Regression...
Evaluating Random Forest Regressor...
Evaluating Gradient Boosting Regressor...
Evaluating Decision Tree Regressor...
Regression evaluation results:
{'Model': 'Linear Regression', 'MSE': 0.9942921103800618, 'RMSE': 0.9971419710252206, 'R2': -3.474314496710278}
{'Model': 'Random Forest Regressor', 'MSE': 0.21873333333333336, 'RMSE': 0.46768935559122293, 'R2': 0.015699999999999936}
{'Model': 'Gradient Boosting Regressor', 'MSE': 0.16574809954729555, 'RMSE': 0.4071217257127106, 'R2': 0.25413355203717003}
{'Model': 'Decision Tree Regressor', 'MSE': 0.3333333333333333, 'RMSE': 0.5773502691896257, 'R2': -0.4999999999999998}
```

Fig. 20 Implementation of model evaluation for regression models, showing metric calculation (MSE, RMSE, R-squared) for assessing performance.

9. Performance Evolution:

As illustrated in Figures 21, classification analysis computes key metrics such as Accuracy, Precision, Recall and F1-Score to measure model performance on training and testing datasets in terms of underfitting, overfitting and generalization. As shown in Figure 22 regression analysis measures metrics like MSE, RMSE and R^2 to gain insights into model performance as well as generalization capabilities, these calculations serve as the foundation of reliability and effectiveness assessments of each model.

```
# Calculate metrics for Train and Test datasets
results = []

for name, model in classification_models.items():
    try:
        # Train predictions
        y_train_pred = model.predict(X_train)
        # Test predictions
        y_test_pred = model.predict(X_test)

        # Train metrics
        train_accuracy = model.score(X_train, y_train)
        train_precision = precision_score(y_train, y_train_pred, average="binary")
        train_recall = recall_score(y_train, y_train_pred, average="binary")
        train_f1 = f1_score(y_train, y_train_pred, average="binary")

        # Test metrics
        test_accuracy = model.score(X_test, y_test)
        test_precision = precision_score(y_test, y_test_pred, average="binary")
        test_recall = recall_score(y_test, y_test_pred, average="binary")
        test_f1 = f1_score(y_test, y_test_pred, average="binary")

        # Append results
        results.append({
            "Model": name,
            "Train Accuracy": train_accuracy,
            "Test Accuracy": test_accuracy,
            "Train Precision": train_precision,
            "Test Precision": test_precision,
            "Train Recall": train_recall,
            "Test Recall": test_recall,
            "Train F1-Score": train_f1,
            "Test F1-Score": test_f1,
        })
    except Exception as e:
        print(f"Could not process {name}: {e}")
        results.append({
            "Model": name,
            "Train Accuracy": np.nan,
            "Test Accuracy": np.nan,
            "Train Precision": np.nan,
            "Test Precision": np.nan,
            "Train Recall": np.nan,
            "Test Recall": np.nan,
            "Train F1-Score": np.nan,
            "Test F1-Score": np.nan,
        })
})
```

Fig. 21 calculating classification metrics (Accuracy, Precision, Recall, F1-Score) for both training and testing datasets

```

# Calculate metrics for Train and Test datasets
results = []

for name, model in regression_models.items():
    try:
        # Fit the model
        model.fit(X_train, y_train)

        # Train predictions
        y_train_pred = model.predict(X_train)
        # Test predictions
        y_test_pred = model.predict(X_test)

        # Train metrics
        train_mse = mean_squared_error(y_train, y_train_pred)
        train_rmse = np.sqrt(train_mse)
        train_r2 = r2_score(y_train, y_train_pred)

        # Test metrics
        test_mse = mean_squared_error(y_test, y_test_pred)
        test_rmse = np.sqrt(test_mse)
        test_r2 = r2_score(y_test, y_test_pred)

        # Append results
        results.append({
            "Model": name,
            "Train MSE": train_mse,
            "Test MSE": test_mse,
            "Train RMSE": train_rmse,
            "Test RMSE": test_rmse,
            "Train R2": train_r2,
            "Test R2": test_r2,
        })
    except Exception as e:
        print(f"Could not process {name}: {e}")
        results.append({
            "Model": name,
            "Train MSE": np.nan,
            "Test MSE": np.nan,
            "Train RMSE": np.nan,
            "Test RMSE": np.nan,
            "Train R2": np.nan,
            "Test R2": np.nan,
        })

# Convert results to a DataFrame
results_df = pd.DataFrame(results)
results_df.to_csv("regression_train_test_metrics.csv", index=False)
print("\nMetrics saved to 'regression_train_test_metrics.csv'.")

```

Fig. 22 Code snippet for training and evaluating regression models, including the calculation of train and test metrics (MSE, RMSE, and R^2) for each model.

10. Results Visualization:

Results visualization for both classification and regression tasks has been designed to facilitate understanding and comparison. Bar plots provide visual comparisons of metrics such as Accuracy, Precision, Recall, F1-Score and AUC-ROC, while confusion

matrices provide insights into predictive capabilities by showing true positives, false positives, true negatives and false negatives as shown in figures 23 and 24.

For regression models, bar plots are used to visualize key metrics, including MSE, RMSE, and (R^2). The MSE comparison indicates the average squared error, showing the accuracy of the predictions. The RMSE comparison provides an interpretable measure of error magnitude, while the R^2 comparison demonstrates the proportion of variance explained by each model as shown in Figures 25 and 26.

Visualizations gives a comprehensive performance evaluation, making it easier to quickly recognize each model's strengths and weaknesses. Performance analysis involves comparing training metrics with testing metrics to determine whether an overfitted, underfitted and well-generalized model exists to an approach which provides invaluable insight into the generalization capabilities and reliability of models.

```
# Save and display results
classification_results_df = pd.DataFrame(classification_results)
classification_results_df.to_csv('classification_results.csv', index=False)
print(classification_results_df.columns)

# Plot performance metrics
classification_results_df.set_index("Model")["Accuracy", "Precision", "Recall", "F1-Score"].plot(kind="bar", figsize=(12, 8))
plt.title("Classification Metrics Comparison")
plt.xlabel("Models")
plt.legend(loc="lower right")
plt.grid(axis="y")
plt.tight_layout()
plt.savefig("classification_metrics_comparison.png")
plt.show()

# Plot confusion matrices
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for idx, (name, cm) in enumerate(confusion_matrices.items()):
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Class 0", "Class 1"])
    disp.plot(ax=axes[idx], colorbar=False)
    axes[idx].set_title(f"Confusion Matrix ({name})")

plt.tight_layout()
plt.savefig("classification_confusion_matrices.png")
plt.show()
```

Fig. 23 Visualizing classification metrics (Accuracy, Precision, Recall, F1-Score, and AUC-ROC) and confusion matrices.

```

# Convert results to a DataFrame
results_df = pd.DataFrame(results)
results_df.to_csv("classification_train_test_metrics.csv", index=False)
print("\nMetrics saved to 'classification_train_test_metrics.csv'.")

# Plot bar charts for each metric
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
for metric in metrics:
    train_values = results_df[f"Train {metric}"]
    test_values = results_df[f"Test {metric}"]
    x = np.arange(len(results_df["Model"])) # Label locations
    width = 0.35 # Bar width

    fig, ax = plt.subplots(figsize=(12, 6))

    # Bar plots
    bars1 = ax.bar(x - width/2, train_values, width, label=f"Train {metric}", color="blue", alpha=0.7)
    bars2 = ax.bar(x + width/2, test_values, width, label=f"Test {metric}", color="orange", alpha=0.7)

    # Add labels, title, and legend
    ax.set_xlabel("Models")
    ax.set_ylabel(metric)
    ax.set_title(f"Train vs Test {metric} Comparison Across Models")
    ax.set_xticks(x)
    ax.set_xticklabels(results_df["Model"], rotation=45, ha="right")
    ax.legend()

    # Annotate bars with values
    for bar in bars1 + bars2:
        height = bar.get_height()
        ax.annotate(f'{height:.2f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), # Offset text by 3 points
                    textcoords="offset points",
                    ha="center", va="bottom")

    # Display and save the plot
    plt.tight_layout()
    plt.savefig(f"train_test_{metric.lower()}_comparison.png")
    plt.show()

```

Fig. 24 visualizing classification metrics (Accuracy, Precision, Recall, F1-Score) using bar plots, comparing training and testing performance across models, with annotations for metric values and legends for clarity.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot Mean Squared Error (MSE)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="MSE", palette="Blues_d")
plt.title("Mean Squared Error (MSE) Comparison Across Models")
plt.ylabel("Mean Squared Error (MSE)")
plt.xlabel("Models")
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("mse_comparison.png")
plt.show()

# Plot Root Mean Squared Error (RMSE)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="RMSE", palette="Oranges_d")
plt.title("Root Mean Squared Error (RMSE) Comparison Across Models")
plt.ylabel("Root Mean Squared Error (RMSE)")
plt.xlabel("Models")
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("rmse_comparison.png")
plt.show()

# Plot R-squared (R2)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="R2", palette="Greens_d")
plt.title("R-squared (R2) Comparison Across Models")
plt.ylabel("R-squared (R2)")
plt.xlabel("Models")
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("r2_comparison.png")
plt.show()
```

Fig. 25 Visualizing Regression metrics (MSE, RMSE, R^2)

```

# Plot bar charts for each metric
metrics = ["MSE", "RMSE", "R2"]
for metric in metrics:
    train_values = results_df[f"Train {metric}"]
    test_values = results_df[f"Test {metric}"]
    x = np.arange(len(results_df["Model"])) # Label locations
    width = 0.35 # Bar width

    fig, ax = plt.subplots(figsize=(12, 6))

    # Bar plots
    bars1 = ax.bar(x - width / 2, train_values, width, label=f"Train {metric}", color="blue", alpha=0.7)
    bars2 = ax.bar(x + width / 2, test_values, width, label=f"Test {metric}", color="orange", alpha=0.7)

    # Add labels, title, and legend
    ax.set_xlabel("Models")
    ax.set_ylabel(metric)
    ax.set_title(f"Train vs Test {metric} Comparison Across Models")
    ax.set_xticks(x)
    ax.set_xticklabels(results_df["Model"], rotation=45, ha="right")
    ax.legend()

    # Annotate bars with values
    for bar in bars1 + bars2:
        height = bar.get_height()
        ax.annotate(f"{height:.2f}",
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), # Offset text by 3 points
                    textcoords="offset points",
                    ha="center", va="bottom")

    # Display and save the plot
    plt.tight_layout()
    plt.savefig(f"train_test_{metric.lower()}_comparison.png")
    plt.show()

```

Fig. 26 Bar plots comparing training and testing regression metrics across models

11. Deployment:

The deployment process entails selecting the best-performing models for classification and regression tasks, taking into account optimal accuracy, interpretability, and computational efficiency. Metrics like Accuracy, Precision, Recall, F1-Score AUC-ROC as well and insight derived from confusion matrices will be taken into consideration during selection for classification models. While for regression models it would be MSE and RMSE as well as R^2 which measure error minimization ability are considered when selecting best-performing regression models, practical considerations like simplicity interpretability and computational efficiency are taken into consideration during selection process.

0.4.3 Power BI Dashboards

Two dashboards were developed in PowerBI to summarize demographic and clinical condition trends:

Dashboards offer a thorough assessment of patient demographics and clinical condition trends. Dashboard 1 as shown in Figure 27 presents gender distribution statistics, in particular, females population slightly more (53.55%) of the total than males population

(46.45%). Studies also demonstrate that older adults, specifically those aged 70-79 years, constitute the greatest proportion of patients for most conditions, followed by 60-69 and 50-59 age groups, younger age groups (such as 12-16 and 17-19 year-olds) represent significantly lower proportions. HIV/AIDS and related conditions (e.g. autoimmune disorders), chronic kidney disease (210,417 patients), and cancer (168,413) are the top five most prevalent health conditions worldwide, accounting for an estimated total population of 957,455. Gender disparity exists for some conditions like autoimmune disorders which impact more females while cancer has more balanced distribution rates.

Dashboard 2 as shown in Figure 28 provides a deeper examination of clinical condition trends across age groups using both a detailed table and a line chart. It reveals that among all age groups, those aged 70-79 were most affected by many conditions affecting the immune system, peaking at 215,661 cases, autoimmune or inflammatory diseases, chronic kidney disease, and blood-borne issues also saw spikes before declining slightly by 80-89 compared with younger populations (12-19). Conversely, rare neurological disorders and chromosomal abnormalities are more prominent among younger populations between these same age range.

These dashboards provide actionable insights into high-risk groups and demographic patterns, acting as essential tools for data-driven healthcare interventions. They highlight the impact of chronic diseases on older populations while simultaneously meeting the unique needs of young groups affected by rare genetic and neurological disorders.

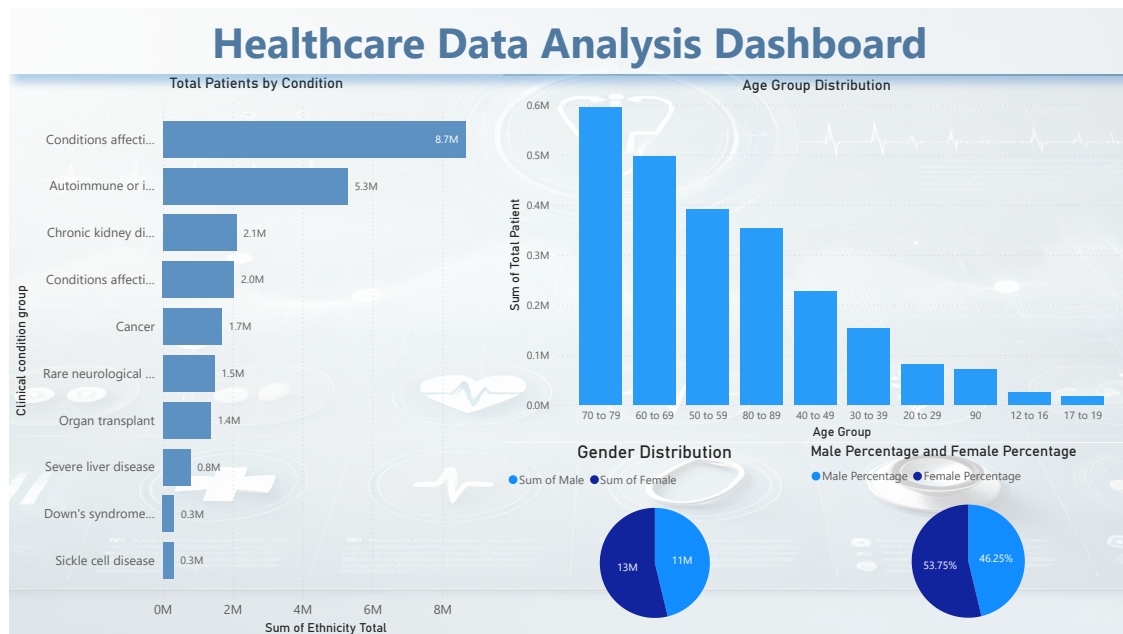


Fig. 27 Power BI visualization summarizing gender, age group, and clinical condition distributions.

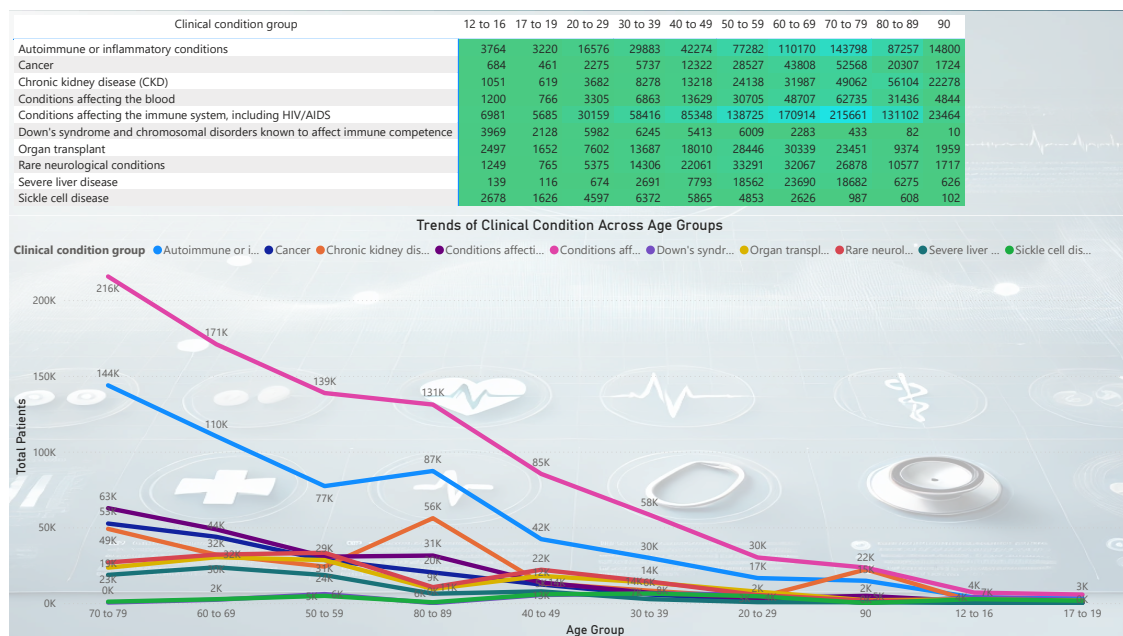


Fig. 28 Power BI visualization illustrating trends of clinical conditions across age groups.

In the following sections address ethical considerations and present results.

0.5 Ethical Consideration

As part of my research, I ensured that ethical considerations were strictly observed during its execution. For this research, the dataset used was obtained from public resources available via the NHS Digital website [2]. This dataset identifies individuals at highest risk from COVID-19 who could qualify for non-hospital-based treatments. All data was anonymized to protect patient confidentiality, according to NHS Digital's terms and conditions. Additionally, appropriate preprocessing and cleaning techniques were implemented to ensure ethical management of sensitive information. Machine learning algorithms such as Logistic Regression, Decision Trees, RF, SVM and K-Means Clustering were employed to analyze the dataset. Analysis and model evaluation processes prioritized fairness, minimized bias, and aligned their findings with ethical guidelines. Results were intended solely for academic and research use based on the principle of beneficence by improving healthcare data analysis techniques while preventing harm to individuals or groups.

0.6 Results

This section assesses machine learning models for health risk stratification by conducting classification and regression tasks. Classification models were assessed against metrics like accuracy, precision, recall, F1 score and AUC-ROC for classification tasks while regression tasks used MSE, RMSE, R^2 Ratios as measures of regression success. Evaluation identifies which machine learning models proved most successful at predicting health risks while outlining both their strengths and limitations.

0.6.1 Classification

Table 5 contains an overview of classification model performance. AB achieved perfect scores across all metrics such as accuracy (1.0), precision (1.0), recall (1.0) and F1-score (1.0). This can be seen further illustrated by Table 29 and Figure 30.

Table 3 Classification Results for Health Risk Stratification Models

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.333	0.0	0.0	0.0
RF	0.667	1.0	0.5	0.667
GB	0.667	1.0	0.5	0.667
Decision Tree	0.667	1.0	0.5	0.667
AB	1.0	1.0	1.0	1.0

Figure 29 provides a detailed comparison between all models’ metrics, revealing AB’s consistently superior performance across all metrics while Logistic Regression struggled significantly, failing to correctly classify any positive instances.

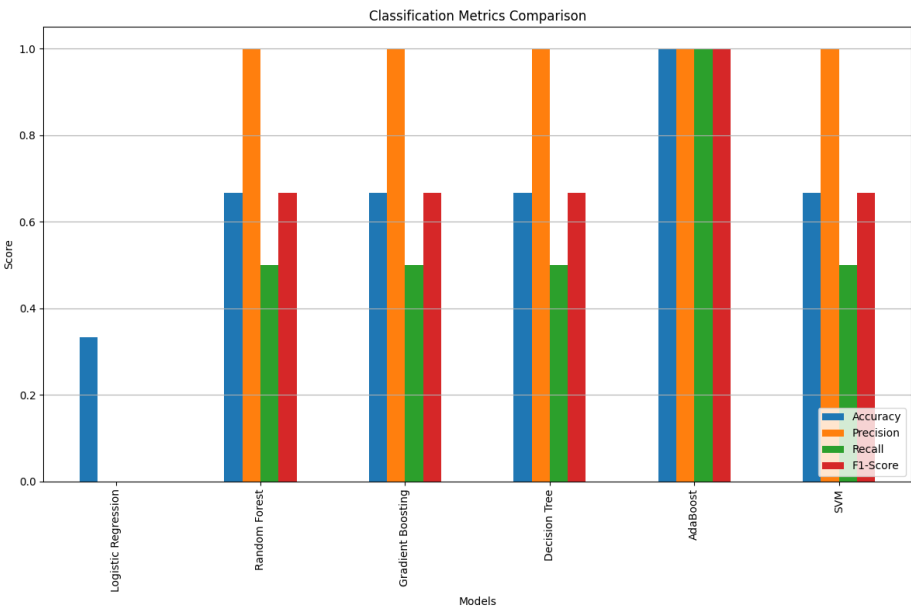


Fig. 29 Comparison of classification metrics (accuracy, precision, recall, and F1-score) across models. AB outperforms all other models.

Figure 30 provides confusion matrices for all models, illustrating their predictions. AB exhibits ideal classification with no misclassifications while Logistic Regression fails to classify positive classes; other models such as RF and GB exhibit misclassifications within positive classes that lead to lower recall values.

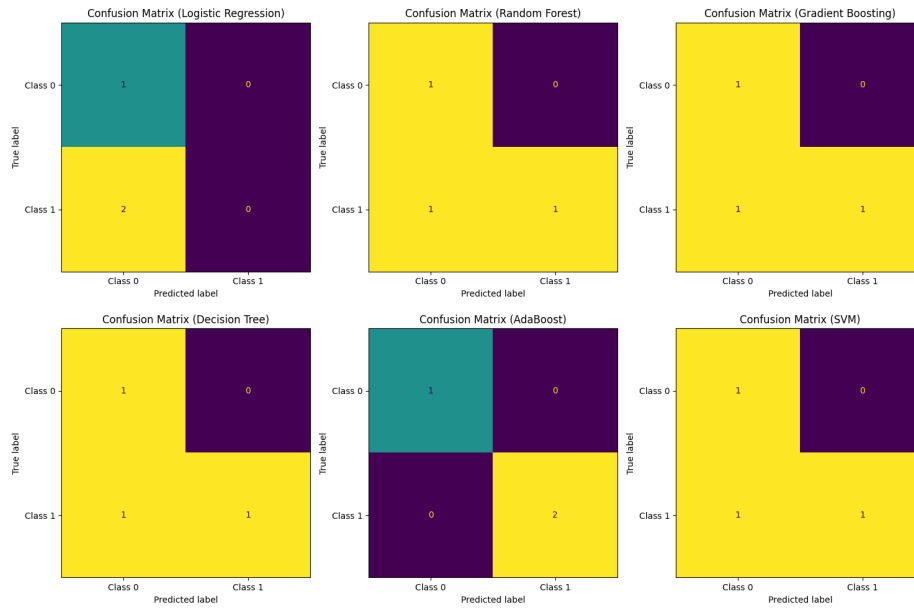


Fig. 30 Confusion matrices for classification models, showing true positives, false positives, true negatives, and false negatives. AB achieves perfect classification, whereas Logistic Regression performs poorly.

0.6.2 Regression

Regression models were evaluated using MSE, RMSE and R^2 scores as evaluation criteria. The Table 4 summarizes the results GB Regressor was identified as the top performer with lowest MSE (0.1657), RMSE (0.4071) and highest R^2 values of 0.2541 to provide accurate predictions and meaningful variance explanation.

Table 4 Regression Results for Health Risk Stratification Models

Model	MSE	RMSE	R-squared (R^2)
LR	0.994292	0.997142	-3.474314
RF Regressor	0.218733	0.467689	0.015700
GB Regressor	0.1657	0.4071	0.2541
Decision Tree Regressor	0.333333	0.577350	-0.500000

Figures 31, 32, and 33 provides a detailed visual comparison of regression model performance metrics.

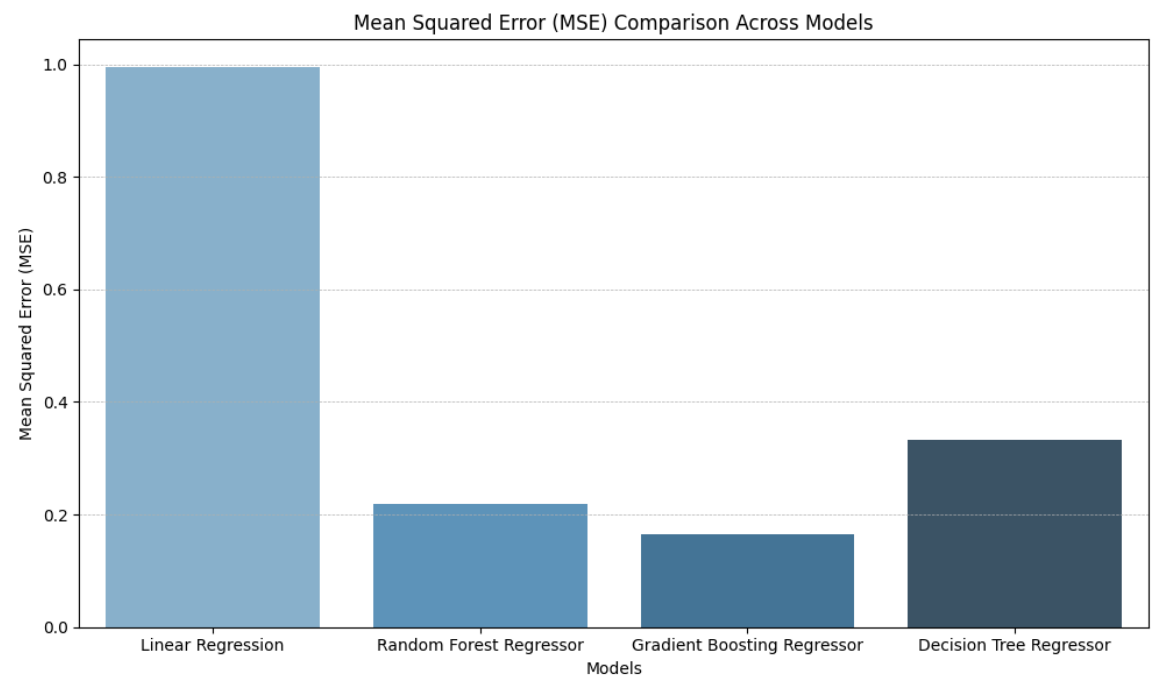


Fig. 31 Comparison of MSE across regression models.

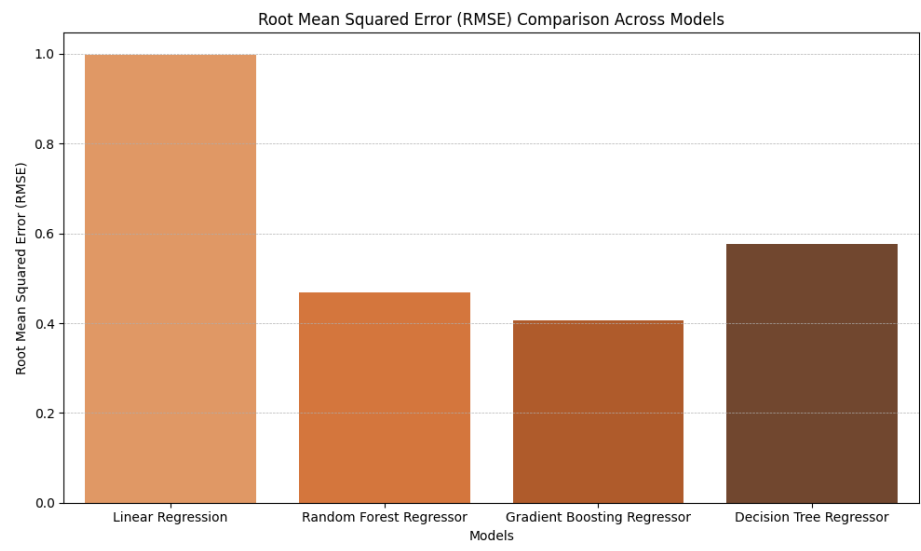


Fig. 32 Comparison of RMSE across regression models.

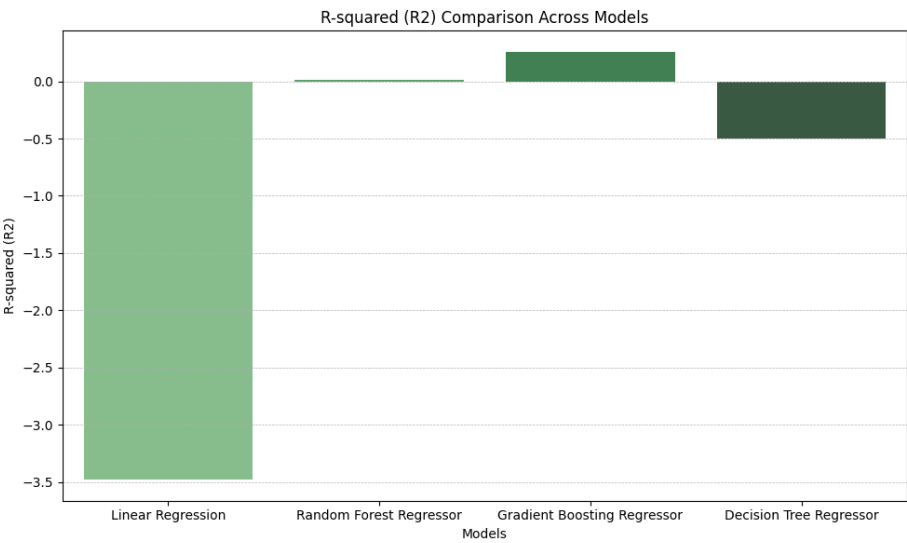


Fig. 33 Comparison of R^2 values across regression models.

0.6.3 Performance Analysis

Performance analysis involves evaluating training and testing metrics of classification and regression models to detect underfitting, overfitting, generalization capabilities.

Classification Models

Table 5, outlines the performance metrics of classification models, while Figures 34, 35, 36, and 37 compare training and testing results respectively.

AB stands out as the superior classification model, attaining perfect scores across all training and testing metrics: Accuracy (1.0), Precision (1.0), Recall (1.0) and F1-Score (1.0). This consistency indicates exceptional generalization without signs of overfitting or underfitting.

RF, GB and Decision Tree models all perform moderately well; all achieve identical Test Accuracy (0.67), F1-Score (0.67) and Precision (0.50). However, their lower Precision estimates reveal some challenge in class prediction balance.

SVM displays an interesting pattern, achieving perfect test metrics while showing decreased Recall (0.5) and F1-Score (0.67), suggesting underfitting during training. Conversely, Logistic Regression displays significant overfitting; its training metrics were perfect while it failed to generalize to its test set.

Table 5 Performance Metrics for Classification Models

Metric	LR	RF	GB	DT	AB	SVM
Train Accuracy	1.00	1.00	1.00	1.00	1.00	0.75
Test Accuracy	0.33	0.67	0.67	0.67	1.00	1.00
Train Precision	1.00	1.00	1.00	1.00	1.00	1.00
Test Precision	0.33	0.50	0.50	0.50	1.00	1.00
Train Recall	1.00	1.00	1.00	1.00	1.00	0.50
Test Recall	1.00	1.00	1.00	1.00	1.00	1.00
Train F1-Score	1.00	1.00	1.00	1.00	1.00	0.67
Test F1-Score	0.50	0.67	0.67	0.67	1.00	1.00

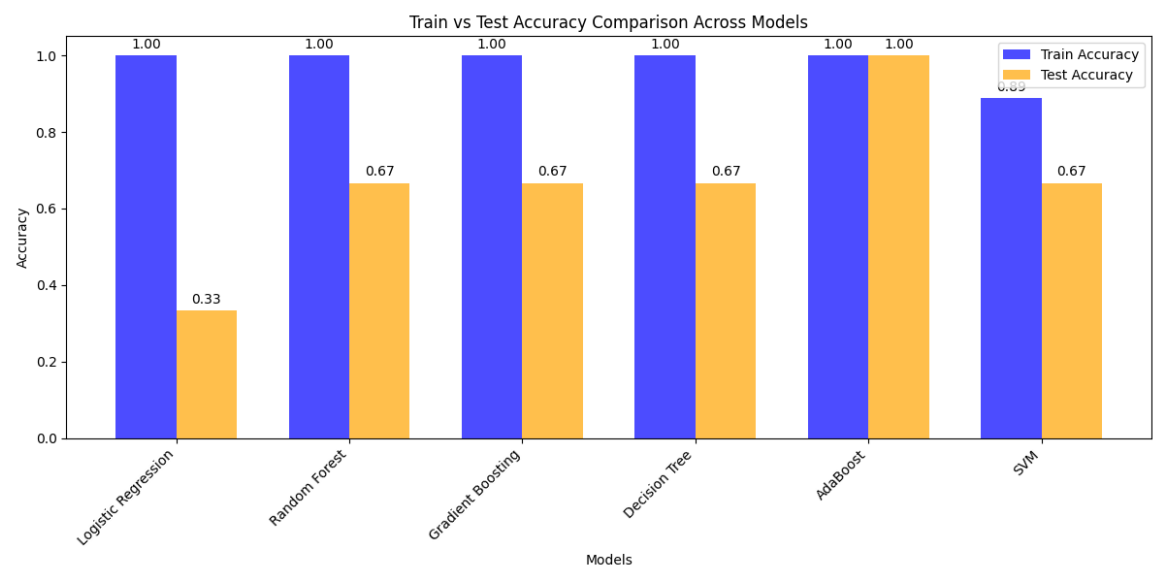


Fig. 34 Train vs Test Accuracy Comparison Across Classification Models

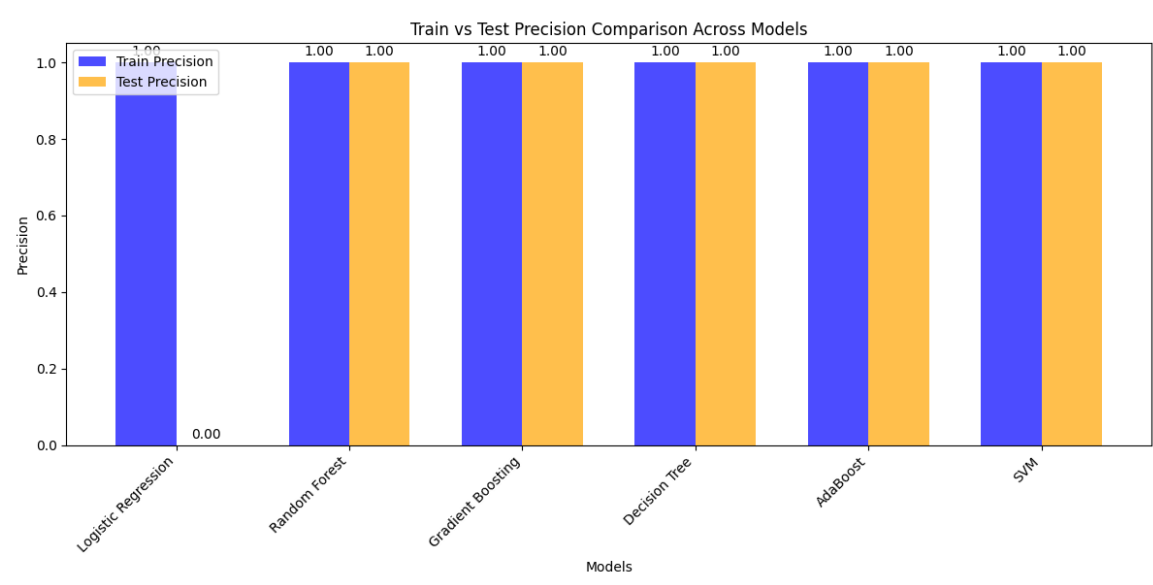


Fig. 35 Train vs Test Precision Comparison Across Classification Models

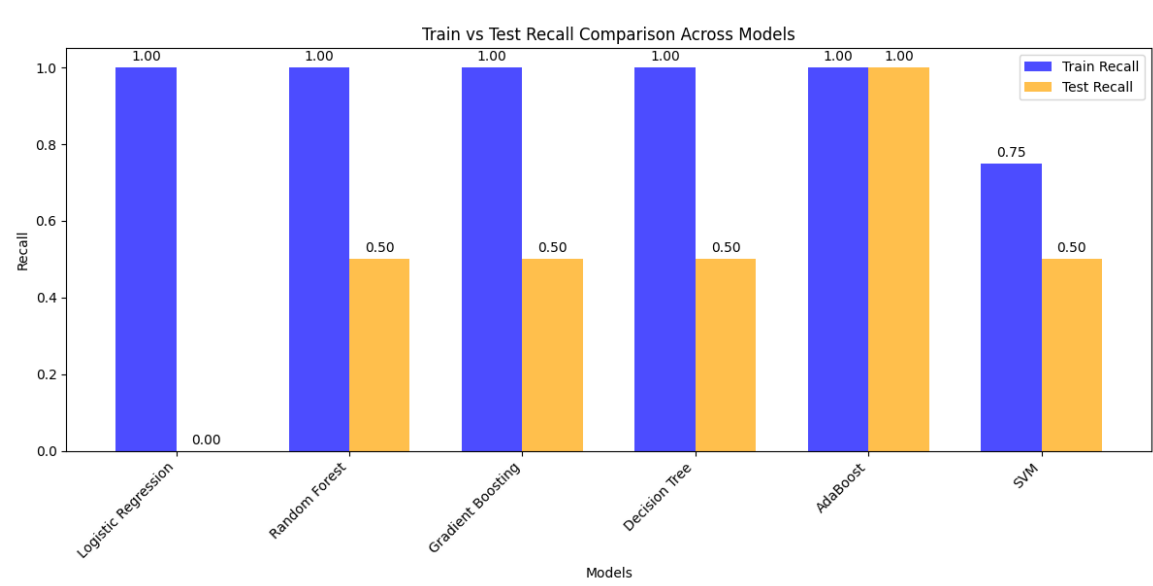


Fig. 36 Train vs Test Recall Comparison Across Classification Models

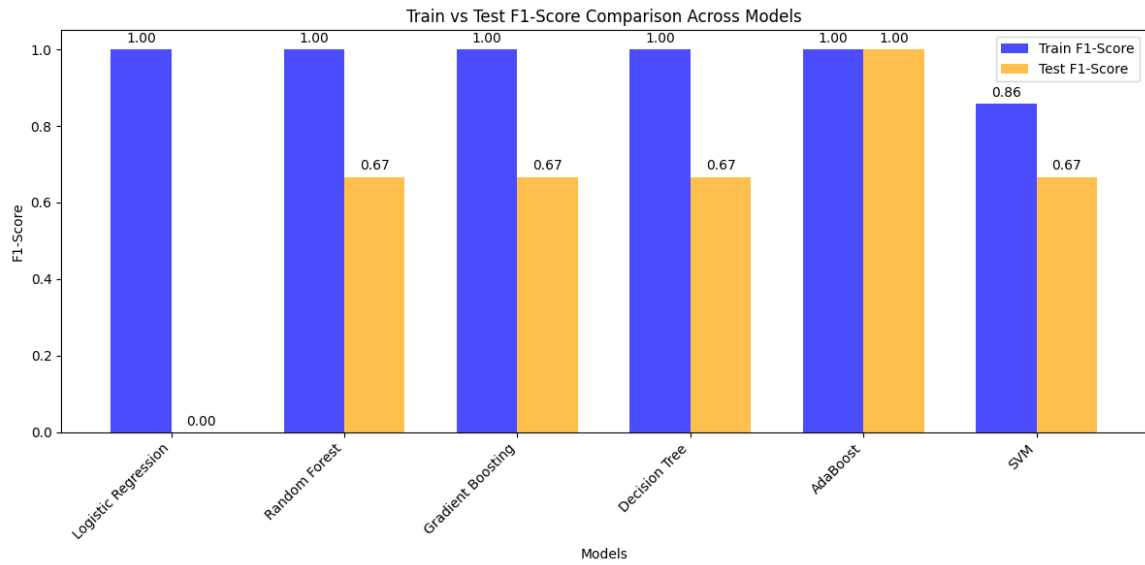


Fig. 37 Train vs Test F1-Score Comparison Across Classification Models

Regression Models

Table 6, Figures 38, 39, and 40 present visual comparisons between training and testing metrics.

GB Regressor found to be the best performing model, by providing a perfect Train $R^2 = 1.0$ as well as the highest Test $R^2 = 0.25$, with the lowest Test RMSE (0.41) these numbers indicate strong predictive accuracy and generalization capabilities.

The RF Regressor performs moderately well with an $R^2 = 0.91$ for Train data and $R^2 = 0.02$ for Test. However, its MSE indicates slight overfitting based on differences between Train and Test RMSE (0.15 vs 0.47).

The Decision Tree Regressor shows signs of overfitting, with its perfect Train $R^2 = 1.0$ but substantially lower Test $R^2 = -0.50$ and high Test RMSE (0.58).

LR exhibits poor generalization, with its train $R^2 = 0.99$ falling sharply below that of its test $R^2 = -3.47$ and having the highest test RMSE of all models (1.00).

Table 6 Regression Model Performance Metrics

Metric	LR	RF	GB	Decision Tree
Train MSE	0.00212	0.02124	1.74×10^{-10}	0.00
Test MSE	0.994	0.219	0.166	0.333
Train RMSE	0.046	0.146	0.000013	0.00
Test RMSE	0.997	0.468	0.407	0.577
Train R^2	0.991	0.914	1.00	1.00
Test R^2	-3.47	0.016	0.25	-0.50

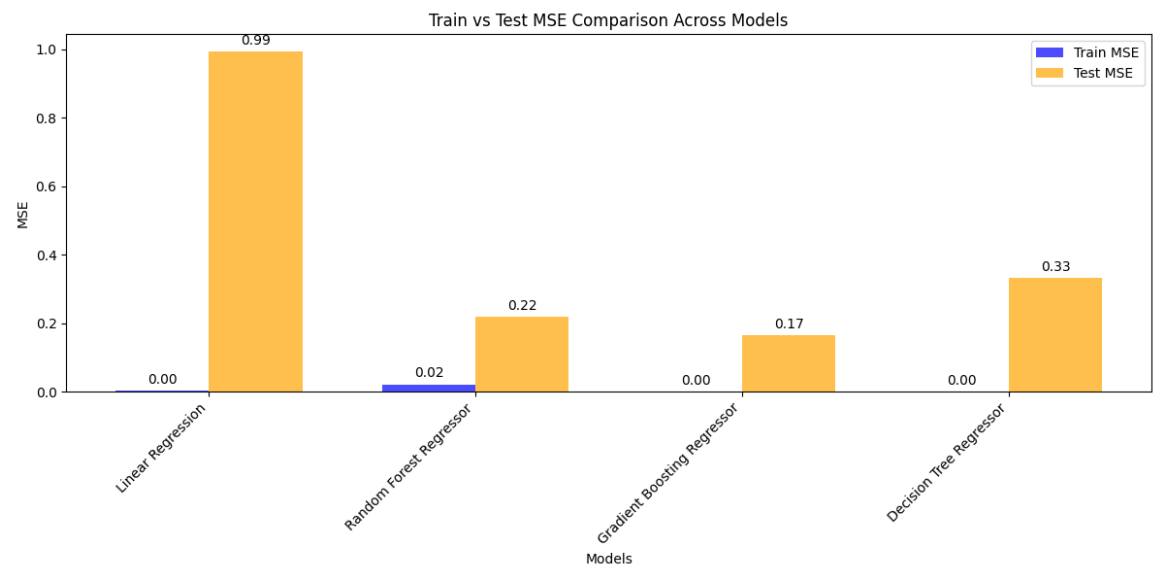


Fig. 38 Train vs Test MSE Comparison Across Regression Models.

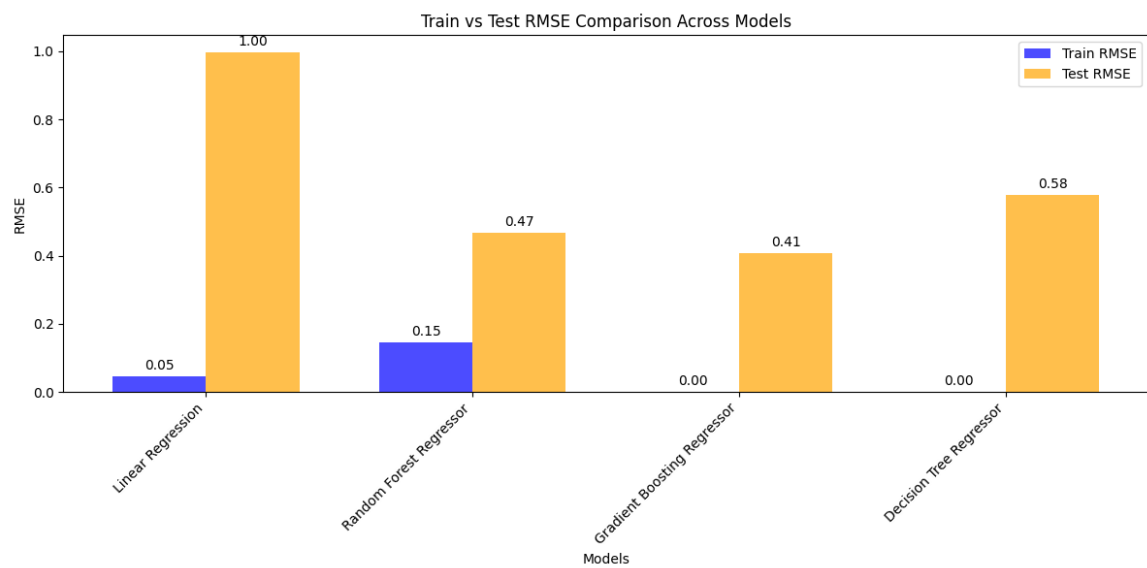


Fig. 39 Train vs Test RMSE Comparison Across Regression Models.

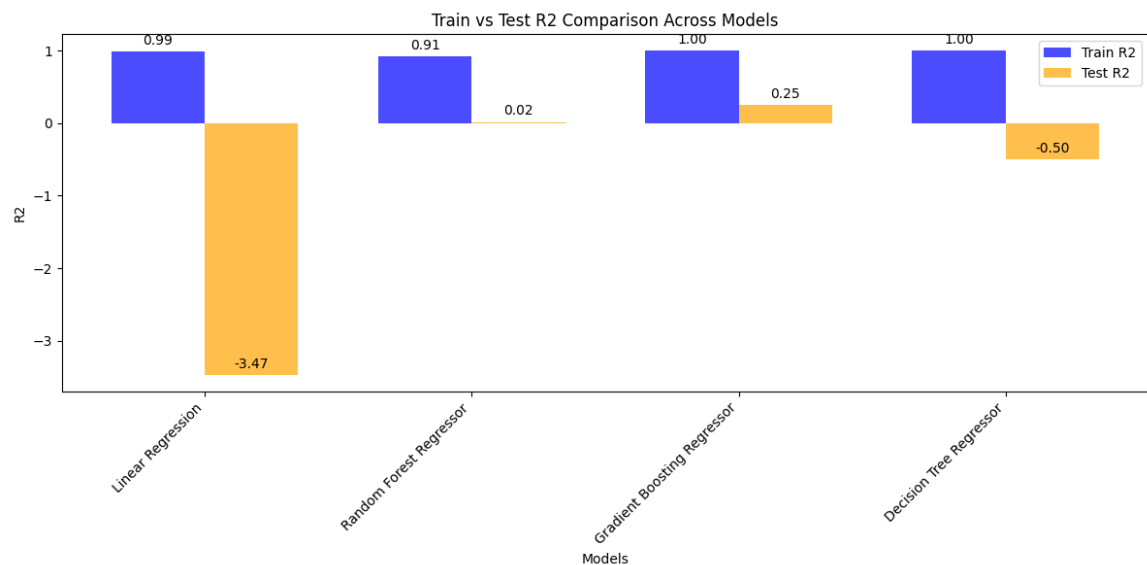


Fig. 40 Train vs Test R^2 Comparison Across Regression Models.

Overall, GB Regressor emerges as the most effective model, showing both predictive accuracy and generalization with remarkable precision. Meanwhile, the RF Regressor performed moderately well but showed slight overfitting, while both Decision Tree and Linear Regression displayed limitations through significant overfitting and poor generalization respectively.

0.7 Conclusion

This research illustrates the importance of machine learning techniques in health-care for identifying high-risk individuals eligible for non-hospital-based COVID-19 treatments, using public data from NHS. Predictive models were then created and rigorously evaluated using comprehensive preprocessing techniques like data cleansing, imputation, feature engineering and class balancing to ensure high-quality analysis.

Initial models' performance evaluation consisted of using key metrics, such as Accuracy, Precision, Recall and F1-Score for classification models, and RMSE, MSE and R^2 for regression models. These calculations were done for both training and test datasets to analyze performance and identify generalized models that performed well on unseen data sets. This approach allowed to detect underfitting or overfitting tendencies while assuring that selected models performed as intended on unseen datasets. Logistic regression and decision tree models demonstrated significant limitations. LR displayed severe overfitting, with a Train $R^2 = 0.994$ but an extremely negative Test $R^2 = -3.47$, suggesting poor generalization. Decision Tree Regressor was equally susceptible, reaching perfect Train $R^2 = 1.0$ while still managing to achieve low Test $R^2 = -0.50$ results. RF Regressor proved moderately effective while showing characteristics with its Train $R^2 = 0.91$ but having low Test $R^2 = 0.016$ results.

From all tested models, the GB Regressor was found to be both well-adapted and highly effective at regression tasks, with its Train $R^2 = 1.0$, and its Test value of $R^2 = 0.254$, MSE value of 0.1657 while RMSE reached 0.4071, demonstrating its strong predictive accuracy as well as generalization capability.

For classification tasks, AB classifier showed excellent generalization with perfect metrics on both train and test datasets: Accuracy = 1.0, Precision = 1.0, Recall = 1.0 and F1-Score = 1.0. Conversely, Logistic Regression displayed severe underfitting, reaching a Train Accuracy of 1.0 while Test Accuracy fell to 0.333.

PowerBI was also utilized for data visualization, providing an effective means of visualizing key patterns and structures present within the dataset. These visualizations helped with exploratory analysis as well as improving interpretability of data, aiding model development and evaluation processes.

This research has profound implications for healthcare decision-making, in particular regarding risk stratification and resource optimization for non-hospital-based COVID-19 treatments. By providing proactive interventions for high-risk individuals, this study shows a scalable framework to improve patient outcomes and operational efficiencies

within healthcare systems. Furthermore, this work serves as the foundation for further advancements such as real-time data stream integrations with advanced analytics as well as expanded data sources that enhance its applicability and impact in addressing public health challenges.

References

- [1] UK Government, “New technology to help identify those at high risk from covid-19,” 2025, accessed: 2025-01-07. [Online]. Available: <https://www.gov.uk/government/news/new-technology-to-help-identify-those-at-high-risk-from-covid-19>
- [2] N. Digital, “Digitally identified individuals at highest risk from covid-19 potentially eligible for non-hospital-based covid-19 treatment in england,” 2024, accessed on December 29, 2024.
- [3] Q.-M. A. Ebugosi and J. A. Olaboye, “Optimizing healthcare resource allocation through data-driven demographic and psychographic analysis,” pp. 1488–1504, 2024. [Online]. Available: https://www.researchgate.net/publication/381720321_Optimizing_healthcare_resource_allocation_through_data-driven_demographic_and_psychographic_analysis
- [4] H. Habebhh and S. Gohel, “Machine learning in healthcare,” *Current Genomics*, vol. 22, pp. 291–300, 2021.
- [5] EIT Health, “Machine learning in healthcare: Uses, benefits, and pioneers in the field,” 2024, accessed: December 30, 2024. [Online]. Available: <https://eithealth.eu/news-article/machine-learning-in-healthcare-uses-benefits-and-pioneers-in-the-field/>
- [6] I. H. Sarker, “Machine learning: Algorithms, real-world applications and research directions,” 2021. [Online]. Available: <https://doi.org/10.1007/s42979-021-00592-x>
- [7] P. K. Deshmukh, “Improving medical image classification using ensemble learning and deep convolutional neural networks,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 4s, pp. 106–121, 2023. [Online]. Available: <https://ijisae.org/index.php/IJISAE/article/view/3756>
- [8] S. Zhou, H. Greenspan, C. Davatzikos, J. Duncan, B. van Ginneken, A. Madabhushi, J. Prince, D. Rueckert, and R. Summers, “A review of deep learning in medical imaging: Imaging traits, technology trends, case studies with progress highlights, and future promises,” *Proceedings of the IEEE Institute of Electrical and Electronics Engineers*, vol. 109, no. 5, pp. 820–838, May 2021. [Online]. Available: <https://doi.org/10.1109/JPROC.2021.3054390>

- [9] X. Chen, H. Chen, S. Nan, X. Kong, H. Duan, and H. Zhu, "Dealing with missing, imbalanced, and sparse features during the development of a prediction model for sudden death using emergency medicine data: Machine learning approach," *JMIR Medical Informatics*, vol. 11, p. e38590, 2023.
- [10] S. Nadakuditi, T. Kumar, and B. Kumar, "Ai and machine learning in healthcare - applications, challenges and ethics," *International Journal of Health Sciences*, vol. 7, 2024.
- [11] I. M. Alkhawaldeh, I. Albalkhi, and A. J. Naswhan, "Challenges and limitations of synthetic minority oversampling techniques in machine learning," *World Journal of Methodology*, vol. 13, no. 5, pp. 373–378, 2023. [Online]. Available: <https://www.wjgnet.com/2222-0682/full/v13/i5/373.htm>
- [12] Y. Yang, H. A. Khorshidi, and U. Aickelin, "A review on over-sampling techniques in classification of multi-class imbalanced datasets: Insights for medical problems," *Frontiers in Digital Health*, vol. 6, p. 1430245, 2024. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fdgth.2024.1430245/full>
- [13] R. S. Abdulsadig and E. Rodriguez-Villegas, "A comparative study in class imbalance mitigation when working with physiological signals," *Frontiers in Digital Health*, vol. 6, p. 1377165, 2024. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fdgth.2024.1377165/full>
- [14] S. M. Foltz, C. S. Greene, and J. N. Taroni, "Cross-platform normalization enables machine learning model training on microarray and rna-seq data simultaneously," *Communications Biology*, vol. 6, p. 222, 2023. [Online]. Available: <https://doi.org/10.1038/s42003-023-04588-6>
- [15] K. A. Linn, B. Gaonkar, J. Doshi, C. Davatzikos, and R. T. Shinohara, "Control-group feature normalization for multivariate pattern analysis using the support vector machine," *UPenn Biostatistics Working Papers*, 2015. [Online]. Available: <http://biostats.bepress.com/upennbiostat/art42>
- [16] K. A. Linn, B. Gaonkar, T. D. Satterthwaite, J. Doshi, C. Davatzikos, and R. T. Shinohara, "Control-group feature normalization for multivariate pattern analysis of structural mri data using the support vector machine," *NeuroImage*, vol. 132, pp. 157–166, 2016. [Online]. Available: <https://doi.org/10.1016/j.neuroimage.2016.02.044>
- [17] D. Singh and B. Singh, "Investigating the impact of data normalization on classification performance," *Applied Soft Computing*, vol. 85, p. 105524, 2019. [Online]. Available: <https://doi.org/10.1016/j.asoc.2019.105524>
- [18] P. Charilaou and R. Battat, "Machine learning models and over-fitting considerations," *World Journal of Gastroenterology*, vol. 28, no. 5, pp. 605–607, 2022. [Online]. Available: <https://www.wjgnet.com/1007-9327/full/v28/i5/605.htm>
- [19] O. Rainio, J. Teuho, and R. Klén, "Evaluation metrics and statistical tests for machine learning," *Scientific Reports*, vol. 14, p. 6086, 2024. [Online]. Available: <https://doi.org/10.1038/s41598-024-56706-x>

- [20] R. Roelofs, “Measuring generalization and overfitting in machine learning,” *University of California, Berkeley Technical Report*, 2019. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-102.html>
- [21] M. Ghanem, A. K. Ghaith, V. G. El-Hajj, A. Bhandarkar, A. de Giorgio, A. Elmi-Terander, and M. Bydon, “Limitations in evaluating machine learning models for imbalanced binary outcome classification in spine surgery: A systematic review,” *Brain Sciences*, vol. 13, no. 12, p. 1723, 2023.
- [22] S. A. Hicks, I. Strümke, V. Thambawita, M. Hammou, M. A. Riegler, P. Halvorsen, and S. Parasa, “On evaluation metrics for medical applications of artificial intelligence,” *Scientific Reports*, vol. 12, p. 5979, 2022. [Online]. Available: <https://www.nature.com/articles/s41598-022-09954-8>
- [23] K. Nagarkar and E. Russo, “Evaluating supervised machine learning classification models in healthcare analytics,” *Milliman White Paper*, 2022. [Online]. Available: https://fr.milliman.com/-/media/milliman/pdfs/2022-articles/12-27-22_evaluating-supervised-ml-classification-models-healthcare.ashx
- [24] G. Varoquaux and O. Colliot, “Evaluating machine learning models and their diagnostic value,” In *Statistical Learning Methods*, New York, NY: Springer, pp. 511–533, 2023. [Online]. Available: https://doi.org/10.1007/978-1-0716-3195-9_20
- [25] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLoS One*, vol. 10, no. 3, p. e0118432, Mar 4 2015.